> **Note**
>
> For those who are familiar with the Databricks Feature Store, you may wonder why we do not utilize it in this chapter. Before Unity Catalog, the Databricks Feature Store provided incredible value with added lineage and metadata details. With the advancement of UC, instead of having a separate Feature Store product, every table utilizing UC automatically has these added benefits. The original Databricks Feature Store has been absorbed into UC.

In the *Applying our learning* section ahead, we will save our features to a feature table using both SQL and `FeatureEngineeringClient`. Once stored, Unity Catalog makes it easy to track the data sources used to create your features and the downstream artifacts (such as models and notebooks) that use each feature. Even better, when you log a model using a training set of features from a feature table, feature metadata is packaged with the resulting model. Watch for this when we are working on the *Streaming Transactions* project.

Feature engineering on a stream

Before diving into feature engineering on a stream, we want to clarify the difference between **streaming pipelines** and **streaming data**. If you have not used Spark Structured Streaming before, it is a stream processing engine built on the Spark SQL engine. It makes it easy to write streaming calculations or transformations like you would write expressions for static data. Structured Streaming pipelines can process batch or streaming data. Streaming pipelines have elements such as checkpoints to automate the data flow. Streaming pipelines, however, are not necessarily always running; rather, they only run when the developer chooses it. In contrast, streaming data (also known as **real-time data**) refers to continuously generated data that can be processed in real time or batch. To simplify, think of streaming pipelines as a series of automated conveyor belts in a factory set up to process items (data) as they come. These conveyor belts can be turned on or off as needed. On the other hand, streaming data is like a never-ending flow of items piling up at the beginning of the line. Depending on the volume of the stream, immediate handling may be necessary. In the context of our *Streaming Transactions* example in the *Applying our learning* section, we are using these automated conveyor belts (streaming pipelines) to efficiently manage and transform this continuous flow of items (streaming data) into a useful form.

Streaming data has several benefits:

- **Immediate insights**: Streaming data allows you to receive quick insights and make real-time decisions based on the data. Speed is essential for applications where timing is critical, such as financial trading or the real-time monitoring of industrial equipment.
- **Up-to-date analysis**: Processing streaming data in real time allows for more current data analysis. Real-time analysis can help you find patterns and trends as they occur and take proactive steps to address potential data quality issues.
- **Improved efficiency**: Streaming data can help organizations become more efficient by enabling them to respond to events quickly and proactively. Short response times can improve customer satisfaction, reduce downtime, and increase productivity.